BARC INTERNSHIP PRESENTATION

Post-Quantum and Classical Cryptography

Theoretical Foundations, Algorithmic Insight, and Comparative Benchmarking

Ishan Malviya · Symbiosis Institute of Technology

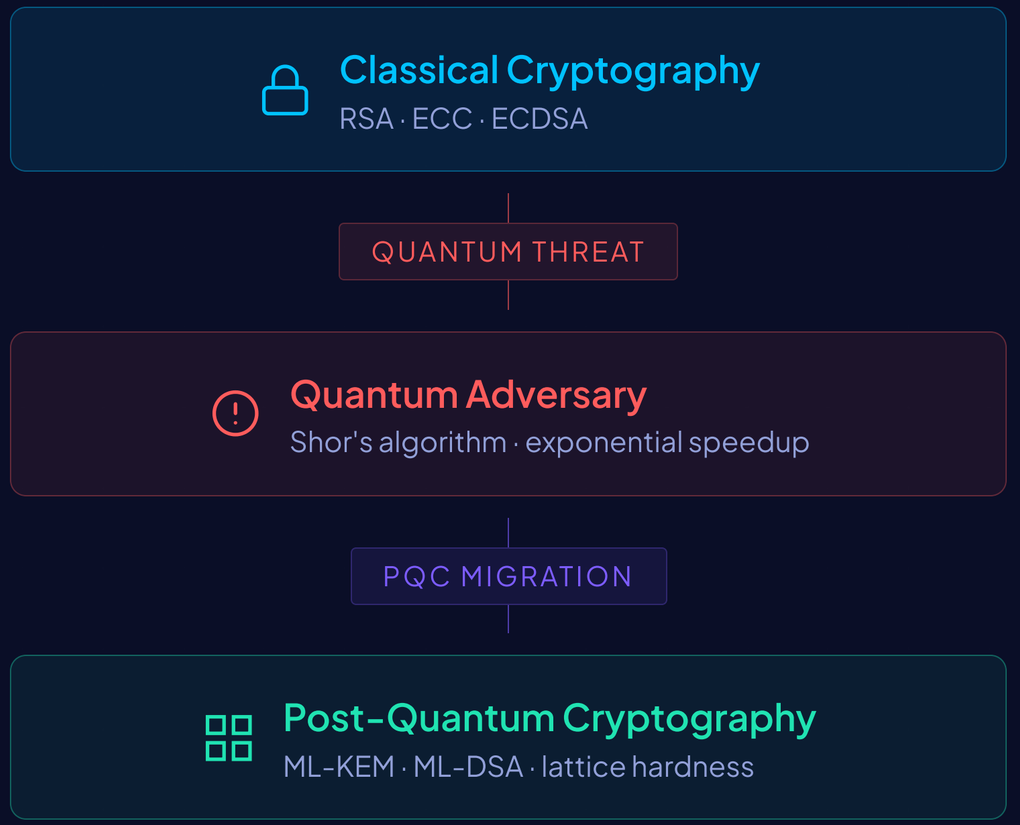
Why Cryptography Is Entering a New Era

Classical public-key security
Rests on integer factoring and discrete logarithm hardness

Quantum computing: the threat
Shor's algorithm breaks these assumptions in polynomial time

Post-quantum cryptography
Replaces classical schemes with lattice-based hardness assumptions

Transition is underway
NIST has standardized ML-KEM and ML-DSA for production use



Question: Can we retain practical security while surviving the quantum shift?

Requirements & Compliance Landscape

Why migration to post-quantum cryptography is not optional

NIST STANDARDISATION TIMELINE

2016	NIST PQC competition launched — 82 candidates submitted
2022	Four finalists selected: Kyber, Dilithium, Falcon, SPHINCS+
2024	FIPS 203/204/205 published — ML-KEM, ML-DSA, SLH-DSA standardised
2030	NSA CNSA 2.0 deadline — all national security systems must migrate

HARVEST-NOW, DECRYPT-LATER

Adversaries are storing encrypted traffic today to decrypt it once a sufficiently large quantum computer exists. Data with a 20–year sensitivity horizon is already at risk.

COMPLIANCE & POLICY DRIVERS

NSA CNSA 2.0	Mandatory PQC for US national security systems by 2030
NIST SP 800-131A	Sunset plan for RSA-2048 and SHA-1 based systems
OMB M-23-02	US Federal inventory and migration roadmap for cryptography
ETSI / EU NIS2	European critical infrastructure quantum readiness obligations

TECHNICAL MIGRATION REQUIREMENTS

- Quantum resistance at NIST Security Level 3 (equivalent to AES-192)
- Backward compatibility during hybrid transition period
- Acceptable key and signature size overhead in constrained environments
- Performance within 10× of classical schemes for most use cases

What Makes This Work Valuable

Four contributions that distinguish this from a textbook survey

Theory to Practice

Connects mathematical foundations directly to real benchmark evidence — not just formulas in a vacuum

Functional Comparison

RSA vs ML-KEM and ECDSA vs ML-DSA compared by cryptographic role — not marketing claims or vendor hype

Two-Track Learning

Toy models for mathematical intuition paired with production libraries for real-world performance realism

Migration Lens

Post-quantum transition is a systems problem — affecting bandwidth, latency, certificates, and protocol design

Research Scope & Methodology

What was implemented, measured, and why it matters

SCHEMES UNDER STUDY

RSA-2048 PKCS #1 v2.2 — Classical

Integer factorisation — keygen, sign, verify

ECDSA-P256 FIPS 186-5 — Classical

Elliptic curve DL — keygen, sign, verify

ML-KEM-768 FIPS 203 — Post-Quantum

Module-LWE — keygen, encap, decap

ML-DSA-65 FIPS 204 — Post-Quantum

Module-LWE — keygen, sign, verify

BENCHMARKING ENVIRONMENT

Language	C (GCC 12) with OpenSSL 3.x and liboqs 0.10
Platform	Linux x86-64, no hardware accelerators disabled
Sampling	1000 independent trials per operation, median reported
Security level	NIST Level 3 — equivalent to AES-192 bit security

METRICS MEASURED

- Key generation time — milliseconds
- Sign / Encap time — milliseconds
- Verify / Decap time — milliseconds
- Public key size — bytes
- Private key size — bytes
- Signature / ciphertext size — bytes

The core novelty: a unified, reproducible head-to-head comparison of classical and PQC primitives under identical conditions.

Mathematical Backbone of Modern Cryptography

Four foundational pillars shared by classical and post-quantum schemes

Entropy and Uncertainty

$$H(X) = -\Sigma\ p(x)\ \log_2\ p(x)$$

Quantifies unpredictability — the foundation of key-space security

Perfect Secrecy

$$P(M \mid C) = P(M)$$

Ciphertext must reveal zero information about the plaintext — Shannon's ideal

Modular Inverse

$$d \equiv e^{-1} \pmod{\varphi(N)}$$

Extended Euclidean algorithm; the basis of RSA private key generation

Security Reductions

$$\text{Break Scheme} \implies \text{Solve } H$$

Breaking a secure scheme implies solving a provably hard problem — the proof technique underlying all modern cryptography

Classical Public-Key Cryptography in One View

N=pq RSA

HARDNESS ASSUMPTION

`Integer Factorisation Problem`

- Large modulus N = pq; security rests on difficulty of recovering p and q
- Conceptually simple structure; operationally costly key generation
- Large key sizes (2048–4096 bits); slower and bulkier than ECC

e, N	d, N	φ(N)
public key	private key	Euler totient

kG ECC / ECDSA

HARDNESS ASSUMPTION

`Elliptic Curve Discrete Logarithm`

- Smaller keys with stronger security per bit than RSA
- Scalar multiplication on curve points is efficient and one-way
- Signature security depends critically on nonce discipline

d	Q = dG	G
private scalar	public point	generator point

Classical systems are elegant and mathematically mature — but both are vulnerable to Shor's algorithm.

RSA: Step-by-Step

1. **Choose large primes p and q**
   `p, q distinct primes, each ≥ 1024 bits`

2. **Compute modulus and totient**
   `N = pq · φ(N) = (p-1)(q-1)`

3. **Choose public exponent e**
   `gcd(e, φ(N)) = 1 · commonly e = 65537`

4. **Compute private exponent d**
   $d \equiv e^{-1} \pmod{\varphi(N)}$ via Extended Euclidean

5. **Encrypt with e, decrypt with d**
   $C = M^e \bmod N$ · $M = C^d \bmod N$

Why Keygen Is Expensive

RSA is conceptually clear, but prime generation requires probabilistic primality testing over very large integers, and big-integer modular exponentiation adds further cost.

PUBLIC KEY

`(e, N)`

PRIVATE KEY

`(d, N)`

RSA

RSA in Practice

Where integer factorisation underpins real-world security

TLS / HTTPS Certificates

X.509 server certificates signed with RSA are the backbone of HTTPS. Certificate authorities issue millions of RSA-2048 and RSA-4096 certs daily.

`X.509 · PKCS #1 v2.2`

Email Encryption & Signing

S/MIME and OpenPGP both use RSA for encrypting session keys and authenticating message origin. Widely deployed in corporate and government email.

`S/MIME · OpenPGP RFC 4880`

SSH Host Authentication

Legacy SSH deployments use RSA host keys and user keys for authentication. ssh-keygen -t rsa still generates RSA-3072 by default in many distributions.

`RFC 4253 · SSH-RSA`

Code Signing

Windows Authenticode, Java JAR signing, and RPM/DEB package signatures use RSA to ensure software authenticity from publisher to end user.

`Authenticode · Jarsigner`

API Token Signing (JWT RS256)

RSA is used in the RS256 and RS512 JWT signing algorithms — common in OAuth 2.0 and OpenID Connect identity providers for stateless API auth.

`RFC 7518 · RS256 / PS256`

Quantum Exposure

Every RSA application above is broken by Shor's algorithm. The security margin of RSA-2048 drops to zero once a sufficiently large fault-tolerant quantum computer exists.

`Shor's → O(log³ N) factoring`

ECDSA: Geometry, Hashing, and Nonce Discipline

1. **Generate key pair**
   `d ← random scalar  ·  Q = dG on curve`

2. **Pick ephemeral nonce k**
   `k ← random in [1, n-1]  ·  must be unique per signature`

3. **Compute commitment point R**
   `R = kG  ·  r = R.x mod n`

4. **Compute signature scalar s**
   $s = k^{-1}(H(m) + rd) \bmod n$

5. **Verify via curve reconstruction**
   $u_1G + u_2Q \rightarrow$ compare x-coordinate with r

Critical Vulnerability

Nonce reuse leaks the private key d algebraically — two signatures with the same k reveal d in closed form.

`Same k → same r → d is recoverable`

SIGNATURE OUTPUT

$\sigma = (r, s)$

P-256: 64 bytes total — efficient and compact compared to RSA-2048's 256 bytes

ECC

ECC & ECDSA in Practice

Compact keys, high throughput — today's dominant asymmetric primitive

TLS 1.3 Key Exchange

ECDH on P-256 and X25519 is the dominant key agreement in TLS 1.3. Every HTTPS connection you make today almost certainly uses ECDH.

`RFC 8446 · X25519 / P-256`

Blockchain Transaction Signing

Bitcoin uses ECDSA on secp256k1 for transaction signatures. Ethereum uses the same for account ownership proofs. Every on-chain action requires a valid ECDSA signature.

`secp256k1 · BIP-340 Schnorr`

FIDO2 / Passkeys / WebAuthn

Hardware security keys (YubiKey, Titan) and platform authenticators (Face ID, Windows Hello) use ECDSA P-256 to sign authentication challenges.

`FIDO2 · W3C WebAuthn L2`

Mobile Device Attestation

Apple Secure Enclave and Android Keystore generate ECDSA P-256/P-384 keys in hardware. Used for app attestation, payments (Apple Pay), and device identity.

`Android Keystore · SEP`

JWT ES256 / API Auth

The ES256/ES384 JWT algorithms use ECDSA P-256/P-384 for signing access tokens in OAuth 2.0 and OpenID Connect. Preferred over RS256 for smaller token size.

`RFC 7518 · ES256 / ES384`

Quantum Exposure

Shor's algorithm solves the elliptic curve discrete logarithm in polynomial time. All ECDH and ECDSA deployments are broken — including blockchain wallets and passkeys.

`Shor's → ECDLP broken in O(n³)`

From Number Theory to Lattices

Shor's algorithm: complete break

Factors integers and computes discrete logarithms in polynomial time — RSA and ECC both fall

Grover's algorithm: partial weakening

Provides quadratic speedup on search — symmetric key lengths must double to compensate

Lattice assumptions: quantum-resistant

No known quantum algorithm achieves meaningful speedup on LWE or SIS problems

A fundamental shift in structure

From elegant low-dimensional algebra to noisy high-dimensional structured geometry

CLASSICAL HARDNESS

$N = pq$	$Q = dG$
Factoring	ECDLP

Broken by Shor's algorithm

↓

LATTICE HARDNESS

$b=As+e$	$Rq=Zq/f$
LWE	Ring-LWE

No known quantum speedup

Lattice Cryptography: Core Mathematical Ideas

Three building blocks that underlie ML-KEM and ML-DSA

LWE SAMPLE

Learning With Errors

$$b = \langle a, s \rangle + e \bmod q$$

INTERPRETATION

Small error term e hides the secret s. Given many (a, b) pairs, recovering s is computationally hard.

$a \in Zq^n$ (random) | e small noise

RING STRUCTURE

Polynomial Ring

$$Rq = Zq[x] / (x^n + 1)$$

INTERPRETATION

Arithmetic over polynomials mod (x^n+1) enables structured, efficient computation that replaces matrix operations.

n = 256 (ML-KEM) | NTT-friendly

STRUCTURED HARDNESS

Module-LWE / SIS

$$t = A \cdot s + e,\ A \in Rq^{k \times k}$$

INTERPRETATION

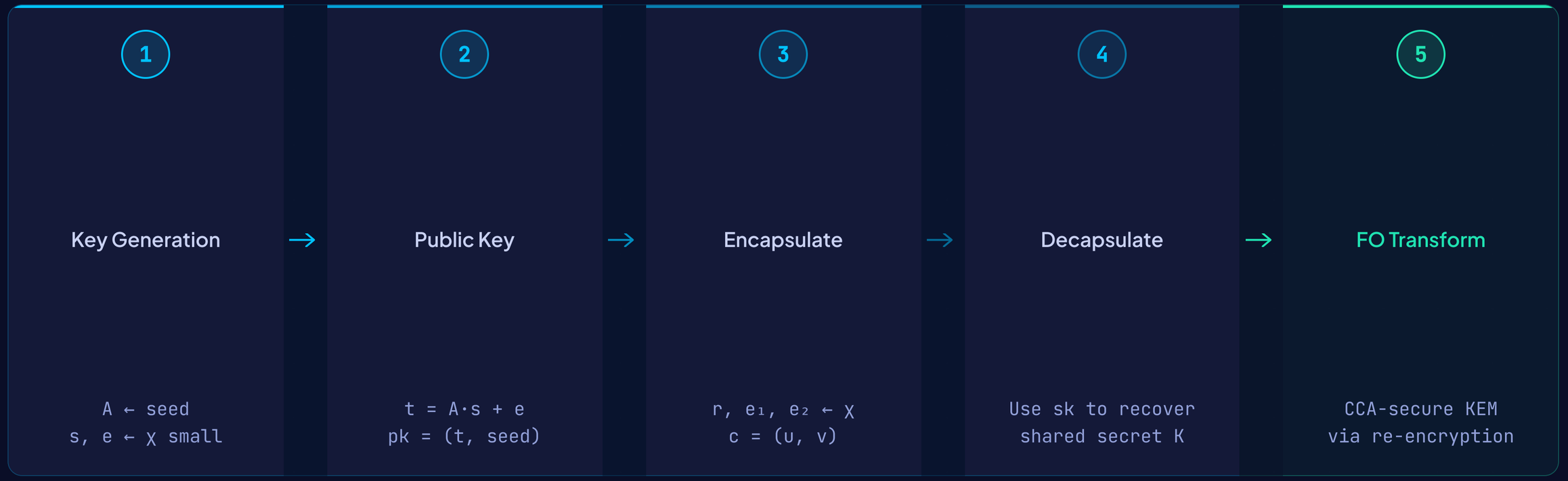
Module structure balances efficiency with security — a single parameter k scales the scheme without redesign.

k=2,3,4 variants | basis of ML-KEM

This is the conceptual bridge to ML-KEM and ML-DSA.

ML-KEM (Kyber): How Quantum-Safe Key Exchange Works

A module-lattice key encapsulation mechanism — NIST FIPS 203



ML-KEM is designed as a practical replacement for Diffie-Hellman-style key establishment.

TLS 1.3 | VPNs | Session keys

KEM

ML-KEM in Practice

FIPS 203 — the post-quantum replacement for Diffie-Hellman key exchange

Hybrid TLS 1.3 Key Exchange

`X25519Kyber768 · IETF draft-ietf-tls-hybrid-design`

The dominant deployment path: combine X25519 (classical) with ML-KEM-768 (PQC) in a single TLS handshake. If either scheme is secure, the session key is secure. Google Chrome and Cloudflare already deploy this.

Post-Quantum VPN / IPsec

`IKEv2 PQC profile · RFC 9370`

IKEv2 has been extended to carry ML-KEM ciphertexts for quantum-safe IKE key establishment. OpenVPN and WireGuard experimental forks support hybrid KEM modes.

Encrypted Messaging Protocols

`Signal PQXDH · X-Wing draft`

Signal Protocol upgraded its X3DH handshake to PQXDH using ML-KEM-1024. The X-Wing hybrid KEM (X25519 + ML-KEM-768) is proposed as a standard building block for messaging apps.

SSH Post-Quantum Key Exchange

`openssh 9.0+ · mlkem768x25519`

OpenSSH 9.0 shipped mlkem768×25519–sha256 as a hybrid KEX method. Many Linux distributions now enable it by default, making SSH handshakes quantum-resistant.

KEY FACTS: ML-KEM-768

Public key	**1,184 bytes**
Private key	**2,400 bytes**
Ciphertext	**1,088 bytes**
Security level	**NIST Level 3**
Standard	**FIPS 203 (2024)**

ADOPTION STATUS

- Google Chrome deployed X25519Kyber768 in 2023
- Cloudflare serves PQ traffic on all edge nodes
- OpenSSH 9.0+ defaults to hybrid KEX
- AWS KMS adding PQC key derivation support

ML-DSA (Dilithium): Signatures Through Commit–Challenge–Response

Fiat–Shamir with aborts — NIST FIPS 204

1. **Sample masking vector y**
   `y ← uniform in [-γ₁, γ₁]ˡ · fresh per signature`
2. **Compute commitment w**
   `w = A·y mod q · w₁ = HighBits(w)`
3. **Derive challenge c**
   `c = H(w₁ ‖ m) · sparse ternary polynomial`
4. **Form response z**
   `z = y + c·s₁ · bound-checked before output`
5. **Reject and restart if bounds fail**
   `‖z‖∞ ≥ γ₁ - β → abort, resample y`

REJECTION SAMPLING

Aborting when z leaks information about s_1 is what makes the scheme secure — the secret remains hidden even after many signatures.

DSA

ML-DSA in Practice

FIPS 204 — post-quantum digital signatures for authentication and integrity

Post-Quantum Code Signing

`FIPS 204 · ML-DSA-65 / ML-DSA-87`

Firmware updates, OS package managers, and application bundles require signatures valid for 10–20 years. ML-DSA replaces ECDSA in signing pipelines where long-term integrity is required.

PKI Certificate Chain Signatures

`X.509 PQC extensions · IETF draft-ietf-lamps-dilithium-certificates`

Certificate Authorities can sign end-entity certificates with ML-DSA. Hybrid dual-signed certificates carry both ECDSA and ML-DSA signatures for backward compatibility during transition.

Government Document Authentication

`NSA CNSA 2.0 mandate`

Legal instruments, identity documents, and archival records require cryptographic signatures that remain verifiable for decades. ML-DSA provides long-term authenticity against future adversaries.

Post-Quantum SSH & TLS Auth

`IETF draft-ietf-tls-sig-scheme`

TLS certificate signatures and SSH host key signatures can be replaced with ML-DSA. This closes the signature half of the quantum threat — complementing ML-KEM for key exchange.

KEY FACTS: ML-DSA-65

Public key	**1,952 bytes**
Private key	**4,032 bytes**
Signature	**3,293 bytes**
Security level	**NIST Level 3**
Standard	**FIPS 204 (2024)**

HYBRID TRANSITION STRATEGY

- Dual-sign with ECDSA + ML-DSA for backward compatibility
- Relying parties that understand ML-DSA validate PQ signature
- Legacy verifiers fall back to the ECDSA component
- No security regression during the migration window

AES & SHA in Practice

The workhorses of every secure system — and why they survive the quantum transition

AES Advanced Encryption Standard

TLS 1.3 Record Layer

`AES-256-GCM`

Every HTTPS payload byte is encrypted with AES-GCM — authenticated encryption providing confidentiality and integrity in a single pass.

Full-Disk & File Encryption

`AES-256-XTS · BitLocker · LUKS · FileVault`

Linux LUKS, Windows BitLocker, and macOS FileVault all use AES-XTS. HashiCorp Vault and age use AES-256 for secrets and archive encryption.

VPN & Wireless Payloads

`AES-256-CBC/GCM · IPsec ESP · WPA3`

IPsec ESP, OpenVPN, and WireGuard encrypt tunnelled traffic with AES. WPA3-Enterprise mandates AES-256-GCMP for Wi-Fi payload protection.

SHA Secure Hash Algorithms

X.509 Certificate Signing

`SHA-256 / SHA-384`

CAs sign certificates over a SHA-256 or SHA-384 digest. Certificate transparency logs, HPKP pins, and trust-store fingerprints are all SHA-256 hashes.

HMAC, HKDF & Key Derivation

`HMAC-SHA256 · HKDF · PBKDF2`

TLS PRF, Signal's HKDF key ratchet, PBKDF2 password hashing, and JWT HS256 MAC all rely on HMAC-SHA-256 as the core PRF primitive.

SHA-3 / Keccak Inside PQC

`SHAKE128 · SHAKE256 · Keccak-f[1600]`

ML-KEM and ML-DSA use SHAKE128 and SHAKE256 internally for hashing, rejection sampling, and PRF operations. SHA-3 is integral to FIPS 203/204.

- **Quantum Safety**

AES-256 retains 128-bit post-quantum security — Grover's algorithm provides only a √N speedup, halving effective key length. **SHA-384/512** likewise maintain ample post-quantum collision resistance. Neither primitive requires replacement — only key-length discipline.

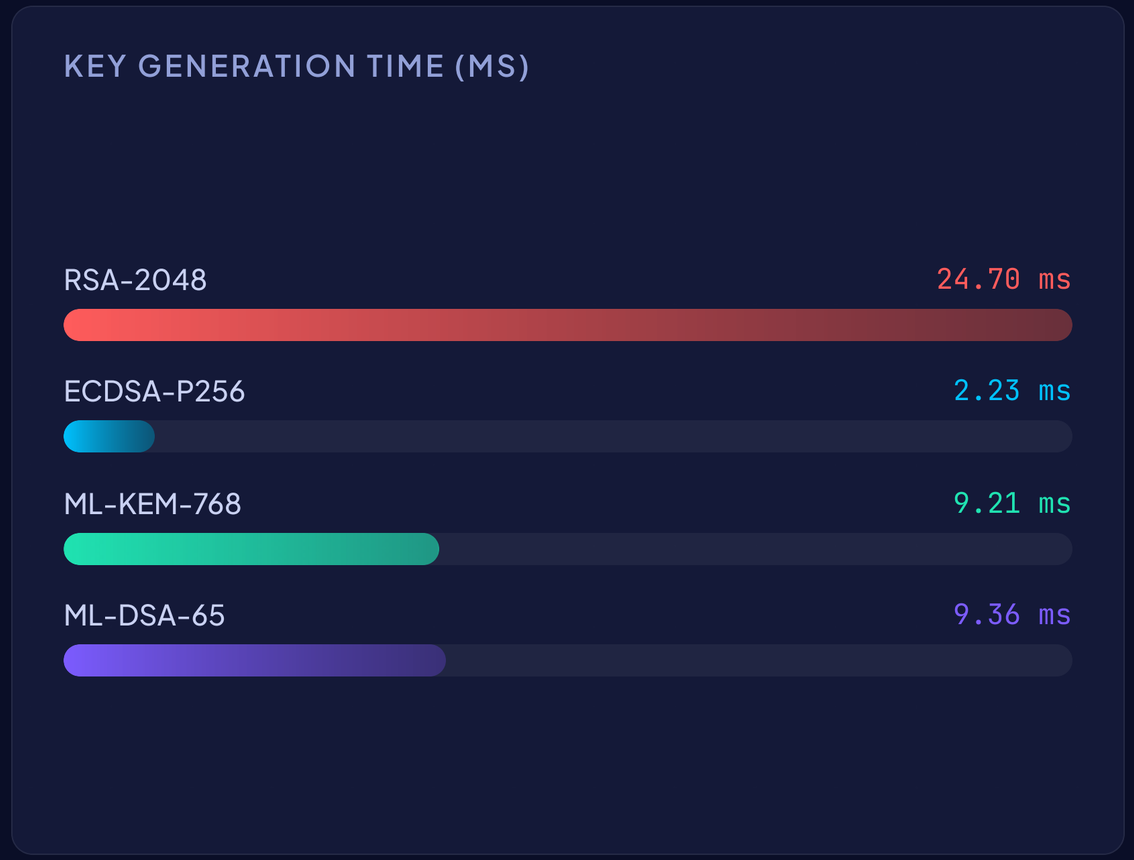
What the Benchmark Results Really Mean

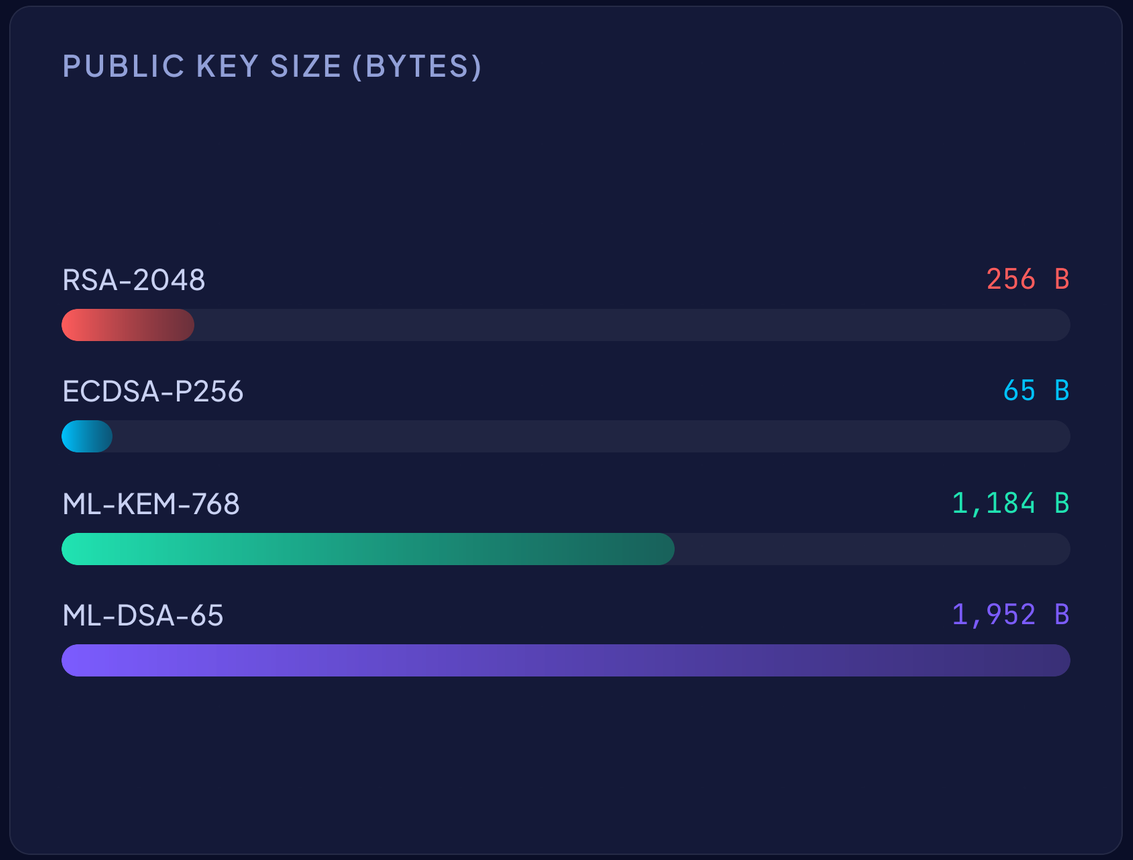
24.7 ms
RSA-2048 keygen
Highest cost — prime generation

2.23 ms
ECDSA-P256 keygen
10× faster than RSA-2048

9.21 ms
ML-KEM-768 keygen
Competitive with RSA, stronger

9.36 ms
ML-DSA-65 keygen
Signature: 3293 bytes





The largest deployment penalty of PQC is often size, not only runtime — certificate chains and handshake payloads grow substantially.

CONCLUSION

Elegant Mathematics, Necessary Transition

I

Classical cryptography remains mathematically elegant and operationally mature

RSA and ECDSA are well-studied, rigorously deployed, and will remain secure in hybrid deployments for the foreseeable future.

II

Post-quantum cryptography is necessary because the threat model is changing

Shor's algorithm changes the threat irreversibly. NIST standardisation of ML-KEM and ML-DSA marks the beginning of a mandated transition, not a speculative one.

III

The future lies in careful migration across security, performance, and protocol integration

Bandwidth, latency, certificate chains, and protocol design all change. The challenge is systems engineering as much as mathematics.

"The real challenge is not inventing stronger mathematics alone — it is engineering the transition responsibly."